.

בית הספר: אורט עמיעד

מסלול: הנדסת תוכנה 883589

מערכת בקרת הורים

Parental Control System

ניטור, הגנת סייבר ובקרת הורים מבוססת רשת מקומית

שם התלמיד:	גור אבידר
ת.ז.:	331751792
שם המנחה:	מריה
שם החלופה:	הגנת סייבר ומערכות הפעלה
שנת לימודים:	תשפ"ה – 2025/2026
תאריך הגשה:	17/03/2026

תוכן עניינים

1. מבוא 3
1.1 ייזום ומטרת הפרויקט 3
1.2 הגדרת הלקוח 5
1.3 הגדרת יעדים ומטרות 6
1.4 בעיות, תועלות וחסכונות 7
1.5 סקירת פתרונות קיימים 9
1.6 סקירה טכנולוגית 11
1.7 תיחום הפרויקט 13
1.8 אפיון המערכת 14
2. מבנה וארכיטקטורה 18
2.1 ארכיטקטורת המערכת 18
2.2 טכנולוגיות ושפות תכנות 20
2.3 זרימת מידע במערכת 22
2.4 פרוטוקול תקשורת 24
2.5 מסכי המערכת 27
2.6 מבני נתונים ובסיס הנתונים 30
2.7 סקירת חולשות ואיומי סייבר 33
3. מימוש הפרויקט 36
3.1 סקירת מודולים ומחלקות 36
3.2 אלגוריתמים מרכזיים 42
3.3 מסמך בדיקות מלא 46
4. מדריך למשתמש 51
5. רפלקציה וסיכום אישי 56
6. ביבליוגרפיה 59
7. נספחים 60

פרק 1: מבוא

1.1 ייזום

1.1.1 תיאור ראשוני של המערכת

אנו חיים בעידן שבו הטכנולוגיה הפכה לחלק בלתי נפרד מחיי היומיום של ילדים ובני נוער. מחקרים עדכניים מצביעים על כך שילד ממוצע בישראל מבלה בין ארבע לשש שעות ביום מול מסך מחשב, ולעתים קרובות אף יותר מכך בסופי שבוע ובחופשות. הגישה לאינטרנט, לרשתות חברתיות, למשחקים מקוונים ולתכנים מגוונים פתחה בפני הדור הצעיר אפשרויות בלתי מוגבלות של ידע, בידור ותקשורת. אולם, עם כל ההזדמנויות שהעולם הדיגיטלי מציע, הוא גם טומן בחובו סיכונים שהורים, מחנכים ואנשי מקצוע מתחום הבריאות הנפשית אינם יכולים להתעלם מהם.

הסיכונים הגלומים בשימוש לא מבוקר במחשב ובאינטרנט הם רבים ומגוונים: חשיפה לתכנים אלימים, פורנוגרפיים או שנאתיים; יצירת קשרים עם גורמים זרים ומסוכנים; פגיעה בהישגים הלימודיים כתוצאה מהסחת הדעת; הפרעות שינה בעקבות שימוש במחשב בשעות הלילה; ובמקרים קיצוניים אף סכנות כגון פגיעה קיברנטית, הטרדה מקוונת והתמכרות לתכנים בעייתיים. הורה מודרני מוצא את עצמו לעתים קרובות חסר אונים מול מציאות זו — הוא יודע שהשימוש במחשב הוא הכרחי, אך אינו יכול לעמוד מאחורי ילדו בכל רגע ורגע.

פרויקט "מערכת בקרת הורים" (Parental Control System) נועד לתת מענה ישיר ומעשי לצורך הזה. המערכת שפותחה במסגרת פרויקט גמר זה היא תוכנת מחשב מלאה, המאפשרת להורה לנטר ולעקוב אחרי פעילות המחשב של ילדו בזמן אמת, ממחשב נפרד, ובאמצעות ממשק גרפי נוח ואינטואיטיבי. המערכת מתבססת על ארכיטקטורת שרת-לקוח (Client-Server) ופועלת ברשת מקומית (LAN) ללא כל צורך בחיבור לשרתים חיצוניים בענן.

הרעיון לפיתוח המערכת הזו עלה מתוך התבוננות ביומיום — ראיתי כיצד הורים מתמודדים עם הקושי הזה, שמעתי סיפורים על מקרים שבהם גילוי בזמן של פעילות בעייתית יכול היה למנוע נזקים משמעותיים. בה בעת, הכרתי את הפתרונות המסחריים הקיימים בשוק וגיליתי שכולם סובלים מאחת מהבעיות הבאות: הם יקרים, הם מחייבים מנוי חודשי, הם תלויים בשרתי ענן חיצוניים שמשמעותם שנתוני ילדיכם עוברים דרך שרתים של חברות זרות, או שהם מציעים ניטור חלקי בלבד. לא מצאתי פתרון חינמי, מקיף ומאובטח שפועל לחלוטין בתוך הבית, ולכן החלטתי לפתח אחד כזה.

1.1.2 רציונל הפרויקט ומה המוצר המוגמר עושה

המוצר המוגמר של פרויקט זה הוא מערכת תוכנה פעילה ומלאה, הכוללת מספר רכיבים עיקריים הפועלים יחד כמכלול אחד מגובש:

הרכיב הראשון הוא שרת מרכזי (server.py) — זהו "לב" המערכת. השרת פועל על מחשב ההורה ומנהל את כל החיבורים הנכנסים. הוא אחראי לאימות זהות המשתמשים, להעברת מסרים בין ההורה לילד, לשמירת נתונים בבסיס הנתונים, ולאכיפת מדיניות האבטחה כולל הגנה מפני מתקפות DDoS. השרת מסוגל לנהל עד 15 חיבורים בו-זמנית ולהגיב לכל אחד מהם בתוך thread נפרד, מה שמבטיח שפעילות של לקוח אחד לא תשפיע על הלקוחות האחרים.

הרכיב השני הוא תוכנת הלקוח (client.py) — זוהי התוכנה המותקנת על מחשב הילד ועל מחשב ההורה. הלקוח תומך בשני מצבי פעולה: מצב גרפי אינטראקטיבי שבו המשתמש מזין פרטי כניסה ידנית, ומצב אוטומטי (headless) שבו הלקוח קורא פרטי כניסה מקובץ טקסט ומתחבר ללא כל התערבות אנושית. מצב אוטומטי זה שימושי במיוחד להרצת בדיקות עומס ולהפעלת הניטור ברקע מבלי שהילד יראה ממשק כניסה.

הרכיב השלישי הוא מנגנון שידור המסך (Screen Sharing) — כאשר ההורה מפעיל אפשרות זו, הלקוח על מחשב הילד מתחיל לצלם את המסך בקצב של 10 פריימים לשנייה, דוחס כל פריים לפורמט JPEG, מקודד אותו ב-Base64, ושולח אותו מוצפן לשרת. השרת מעביר את הנתונים לממשק ההורה, שם הם מוצגים על גבי קנבס גרפי בזמן אמת. כל הפעולות הללו מתרחשות ברקע, ללא הפרעה לפעילות הרגילה של הילד על המחשב.

הרכיב הרביעי הוא מנגנון ניטור המקלדת (Keylogger) — כאשר ההורה מפעיל מאזין זה, תוכנת הלקוח על מחשב הילד מפעילה האזנה ברמת מערכת ההפעלה לכל הקשה על המקלדת. ההקשות מצטברות ונשלחות בחבילות של 20 תווים או כל 0.8 שנייה לשרת, שמעביר אותן לממשק ההורה. בסיום הניטור, כל ההקשות שנאספו נשמרות לקובץ טקסט בתיקייה ייעודית ומתועדות בבסיס הנתונים.

הרכיב החמישי הוא מנגנון ההצפנה (Encryption) — כל תקשורת בין הלקוחות לשרת מוצפנת באמצעות אלגוריתם AES-GCM, שהוא תקן ההצפנה המודרני המומלץ על ידי ה-NIST. ההצפנה מבטיחה שגם אם מישהו ברשת המקומית ינסה ליירט את התקשורת, הוא לא יוכל לקרוא את תוכן ההודעות ולא יוכל לשנות אותן מבלי שהדבר יתגלה.

הרכיב השישי הוא מנגנון הגנת DDoS — המערכת מיישמת שני מנגנוני הגנה מקבילים: מגבלת חיבורים כוללת (מקסימום 15 חיבורים בו-זמנית) ומגבלת חיבורים לפי כתובת IP (מקסימום 3 חיבורים מאותה כתובת). כאשר לקוח מנסה לחרוג ממגבלות אלו, השרת לא רק מנתק אותו, אלא גם מנתק את כל החיבורים הקיימים מאותה כתובת IP, מסמן את המשתמש כמשתמש חשוד בבסיס הנתונים, ומונע ממנו להתחבר מחדש בעתיד.

הרכיב השביעי הוא מסך הפתיחה (Splash Screen) — כאשר מפעילים את השרת, מסך פתיחה מרשים מוצג תחילה: וידאו באיכות גבוהה עם מוזיקה, ולאחר מכן השרת עולה ומתחיל לפעול. מסך הפתיחה מציג את הלוגו של המערכת ואת שמה, ויוצר רושם ראשוני מקצועי.

הרכיב השמיני הוא תוכנית הרצה מרובת לקוחות (multi_run.py) — תוכנית זו מפעילה בו-זמנית 20 threads, כל אחד מהם מריץ עותק של תוכנת הלקוח עם פרטי כניסה שונים. תוכנית זו שימשה כלי מרכזי בשלב הבדיקות לוידוא שהשרת עמיד תחת עומס.

1.1.3 למה בחרתי בפרויקט זה

הבחירה בפרויקט מערכת בקרת הורים לא הייתה מקרית — היא נבעה מצומת של מספר גורמים שהצטלבו עבורי:

הגורם הראשון הוא הרלוונטיות החברתית. ניטור הורים הוא נושא שמשפיע ישירות על חיי אנשים רבים. ראיתי כיצד הורים בסביבתי מתמודדים עם חוסר הוודאות לגבי מה ילדיהם עושים בשעות הארוכות שהם מבלים מול המחשב. הפרויקט הזה אינו תרגיל אקדמי גרידא — הוא פתרון לבעיה אמיתית.

הגורם השני הוא אתגר טכני משמעותי. הפרויקט דרש ממני לשלב ידע מתחומים רבים: תקשורת רשת, הצפנה, ממשקי משתמש, בסיסי נתונים, עיבוד תמונה ועוד. זה בדיוק הסוג של פרויקט שמאלץ אותך לצאת מאזור הנוחות שלך וללמוד דברים חדשים — וזה הסוג של אתגר שמצאתי בו עניין אמיתי.

הגורם השלישי הוא הקשר לתחום הסייבר. לימודי בהתמחות הגנת סייבר, ופרויקט שמשלב תקשורת מוצפנת, הגנה מפני מתקפות ואבטחת מידע הוא ביטוי מושלם של מה שלמדתי לאורך שנות לימודי. זה איפשר לי להשתמש בידע התיאורטי שצברתי ולהפוך אותו לפתרון פרקטי ועובד.

הגורם הרביעי הוא הפוטנציאל להרחבה עתידית. המערכת שבניתי היא בסיס איתן שעליו ניתן להמשיך לבנות. בעתיד ניתן להוסיף יכולות כמו ניטור שמע, סינון אתרים אוטומטי, התראות SMS, ממשק web ועוד. הבניתי את המערכת עם מחשבה על הרחבה, עם ממשקים ברורים בין הרכיבים.

1.1.4 אתגרים שצפיתי לעצמי

בתחילת הפרויקט, לפני שכתבתי שורת קוד אחת, ישבתי וחשבתי על האתגרים שאני צפוי להתמודד איתם. ציפיתי לאתגרים הבאים:

האתגר הראשון שציפיתי לו היה ניהול מקבילות (Concurrency). שרת שצריך לשרת מספר לקוחות בו-זמנית הוא לא דבר פשוט. חשבתי על הצורך לנהל threads מרובים, על בעיות של race conditions ועל הצורך לסנכרן גישה למשאבים משותפים כמו בסיס הנתונים ומילון הלקוחות המחוברים. לא שיערתי עד כמה הבעיה הזו תהיה עמוקה — ובמיוחד לא ציפיתי לבעיית thread-safety של Tkinter שתעסיק אותי שבוע שלם.

האתגר השני היה שידור מסך בזמן אמת. צילום מסך, דחיסה, קידוד, שליחה, קבלה, פענוח ותצוגה — כל זאת ב-10 פריימים לשנייה — נשמע כמו הרבה פעולות. חששתי מבעיות ביצועים, מהשהיה גבוהה ומניצול משאבי מעבד מוגזם. בסוף הצלחתי להשיג קצב תצוגה סביר עם שימוש ב-JPEG quality=50 ורזולוציה מוקטנת של 480×270 פיקסלים.

האתגר השלישי היה ההצפנה. הכרתי את המושג הצפנה אך מעולם לא יישמתי אותה בפועל בפרויקט אמיתי. לא ידעתי כיצד להגדיר מפתח, כיצד לנהל את ה-Nonce, מה ההבדל בין CBC ל-GCM, וכיצד לשלב את ההצפנה עם פרוטוקול שליחה שמבוסס על אורך הודעה ב-4 bytes.

האתגר הרביעי היה עיצוב ממשק המשתמש. לא רציתי ממשק גנרי ומכוער. רציתי ממשק שנראה מקצועי, עם ערכת צבעים עקבית, כפתורים מעוצבים ומחוונים ויזואליים שמספרים להורה מה קורה בכל רגע. אך Tkinter הוא כלי בסיסי יחסית לעיצוב UI מתקדם, ולכן ידעתי שאצטרך לכתוב הרבה קוד עיצוב ידני.

האתגר החמישי היה מנגנון ה-DDoS. הגנה מפני DDoS היא תחום שלם בפני עצמו. ידעתי שאני צריך לנהל סטטיסטיקות חיבורים בזמן אמת, לאכוף מגבלות, ולדאוג שהפעולות האלה תהיינה thread-safe. לא ציפיתי שיהיה קשה — אבל כאשר כתבתי את הקוד גיליתי שיש מקרים קצה עדינים שצריך לטפל בהם בזהירות.

1.2 הגדרת הלקוח

1.2.1 למי מיועדת המערכת

כאשר מדברים על "לקוח" בהקשר של פרויקט תוכנה, אנחנו מתכוונים לאנשים שישתמשו במערכת ואשר הצרכים שלהם הכתיבו את ההחלטות התכנוניות שנלקחו לאורך הפיתוח. בפרויקט זה, ניתן לזהות מספר קבוצות משתמשים ברורות, שלכל אחת מהן פרופיל, צרכים ויכולות שונים.

1.2.2 משתמש ראשי — ההורה

ההורה הוא המשתמש המרכזי של המערכת מבחינת שליטה וסמכות. זהו האדם שמתקין את השרת על מחשבו, מנהל את החשבונות במערכת, ומשתמש בממשק הניטור כדי לעקוב אחרי פעילות ילדו. מאפיינים אופייניים של משתמש זה:

מבחינת גיל וידע טכנולוגי — ההורה הטיפוסי הוא בגיל 30–55, עם ידע טכנולוגי ברמה בינונית עד נמוכה. הוא יכול להוריד ולהתקין תוכנות, אבל לא בהכרח מכיר מושגים כמו "socket" או "thread". לכן, ממשק המשתמש שתוכנן עבורו הוא פשוט, ישיר ואינטואיטיבי — עם כפתורים גדולים ומסומנים בצורה ברורה, ומחוונים ויזואליים שמספרים לו בכל עת מה מצב הניטור (פעיל/לא פעיל).

מבחינת הצרכים — ההורה רוצה לדעת מה ילדו עושה במחשב, ובמיוחד: האם הוא גולש באתרים בלתי מתאימים? האם הוא מתכתב עם אנשים לא מוכרים? האם הוא משחק בשעות הלילה כשאמור לישון? האם הוא מכין שיעורים או גולש ברשתות חברתיות? המערכת עונה על כל השאלות הללו באמצעות שידור המסך ורישום המקלדת.

מבחינת מגבלות — ההורה אינו יכול לשבת כל היום ולצפות במסך של ילדו. לכן, המערכת תוכננה גם לשמור נתונים: יומני המקלדת נשמרים לקבצים ולבסיס הנתונים, כך שההורה יכול לעיין בהם בנוחות בכל עת. בנוסף, ממשק ההורה כולל שעון חי ומחוונים של סטטוס כדי שיוכל לפקח ולצפות בנוחות.

מבחינת תנאי הפעלה — ההורה מחובר לאותה רשת מקומית (Wi-Fi ביתי או קווי LAN). אין צורך בתצורה מיוחדת של רשת — פשוט מפעילים את השרת על מחשב ההורה ואת הלקוח על מחשב הילד, ושניהם באותה רשת.

1.2.3 משתמש משני — הילד

הילד הוא הצד המנוטר במערכת. תוכנת הלקוח מותקנת על מחשבו ופועלת ברקע, שולחת נתונים לשרת. מאפיינים אופייניים:

מבחינת גיל ואוריינות טכנולוגית — הילד הטיפוסי הוא בגיל 8–17. מעניין לציין שדווקא הילדים המבוגרים יותר (14–17) לרוב בעלי ידע טכנולוגי גבוה יותר מהוריהם, ולכן נדרשה מחשבה מיוחדת על כך שהם לא יוכלו לזהות בקלות את פעילות הניטור ולעצור אותה.

מבחינת ממשק — ממשק הילד הוא מינימלי ביזוד. לאחר הכניסה למערכת, הוא רואה רק מסך פשוט עם כיתוב "SESSION ACTIVE" ונקודה ירוקה מהבהבת. אין לו גישה לפקדים של ניטור, אין לו אפשרות לכבות את השידור, ואין לו מידע על מה בדיוק מנוטר ברגע זה.

מבחינת שמירת פרטיות — יש לציין כי השימוש במערכת מחייב שקיפות ותקשורת פתוחה בין הורים לילדים. המערכת היא כלי טכנולוגי, אך ההיבט החינוכי-תקשורתי של שיחה עם הילד על הסיבות לניטור הוא חיוני לא פחות מהטכנולוגיה עצמה.

1.2.4 משתמשים פוטנציאליים נוספים

מעבר לשימוש הביתי, המערכת מתאימה לשימוש במגוון הקשרים נוספים:

מנהלי מעבדות מחשבים בבתי ספר — בית ספר שמפעיל מעבדת מחשבים יכול להשתמש במערכת כדי לנטר את עמדות העבודה בזמן שיעורים. המורה יכול מהמחשב שלו לצפות בכל מה שהתלמידים עושים ולוודא שהם עובדים על המטלות ולא גולשים לאתרים לא קשורים. בהקשר זה, היכולת לנהל מספר חיבורים בו-זמנית (עד 15) היא בדיוק מה שדרוש.

עסקים קטנים — מנהל עסק שמעוניין לנטר את עמדות העבודה של עובדיו יכול להשתמש במערכת. שימוש כזה מחייב כמובן הסכמה מלאה של העובדים ועמידה בדיני העבודה הרלוונטיים, אבל מבחינה טכנית המערכת מסוגלת לשמש לצורך זה.

מוסדות שיקום וטיפול — מוסדות שמאכלסים בני נוער עם בעיות התנהגותיות עשויים להזדקק לניטור הדוק יותר של פעילות המחשב. המערכת מאפשרת ניטור מרוחק מבלי שהאדם המנוטר צריך להיות מודע לכך בכל רגע.

1.3 הגדרת יעדים ומטרות

1.3.1 מטרות ראשוניות — חיוניות להצלחת הפרויקט

מטרות ראשוניות הן אלו שללא מימושן הפרויקט לא יוכל להיחשב מוצלח. מדובר בפונקציונליות הגרעינית שהגדרתי לעצמי בתחילת הדרך:

מטרה ראשית ראשונה: שידור מסך בזמן אמת. ההורה חייב להיות מסוגל לפתוח חלון בממשק שלו ולראות בו מה מוצג כרגע על המסך של ילדו. הדרישה המינימלית שהגדרתי לעצמי הייתה קצב של לפחות 10 פריימים לשנייה, שהם מספיקים לצפייה נוחה אך אינם עמוסים על הרשת המקומית. הדרישה נוספת הייתה שהשידור יעבוד באמינות ובלא קריסות, גם לאחר זמן ממושך של פעולה.

מטרה ראשית שנייה: ניטור מקלדת. כל הקשה שהילד מקיש על המקלדת תועבר לממשק ההורה ותישמר לצמיתות. המטרה הייתה שהנתונים יגיעו להורה תוך פחות משנייה מרגע ההקשה, ושיישמרו בצורה מסודרת ומתועדת עם חותמות זמן.

מטרה ראשית שלישית: הצפנת תקשורת מלאה. כל בית הנתונים שעובר בין השרת ללקוחות מוצפן בלא יוצא מן הכלל. אין הודעה אחת שנשלחת בטקסט גלוי. מטרה זו הכרחית לא רק מבחינת אבטחה אלא גם מבחינת פרטיות — נתוני הניטור (תמונות מסך, הקשות מקלדת) הם נתונים רגישים ביותר שאסור לאפשר לגורמים שלישיים לגשת אליהם.

מטרה ראשית רביעית: ניהול משתמשים מלא. המערכת חייבת לאפשר הרשמה, כניסה, אימות ותחזוקה של חשבונות משתמשים. הסיסמאות נשמרות כ-hash בלבד, הצמדת הורה-ילד מנוהלת בבסיס הנתונים, וכל חיבור מאומת לפני שמתאפשרת גישה לאיזו פעולה שהיא.

מטרה ראשית חמישית: יציבות ועמידות. השרת חייב לפעול ברציפות ללא קריסות, גם כאשר לקוחות מתנתקים פתאום, גם כאשר נשלחות הודעות לא תקינות, וגם כאשר ניסיונות חיבור חריגים מתרחשים. כל חריגה נתפסת ב-try/except מתאים, ומשאבים מנוקים כראוי ב-finally.

1.3.2 מטרות משניות — מעלות את איכות המוצר

מטרות משניות הן פונקציונליות שתרמה מאוד לאיכות הסופית של המוצר, אך היה ניתן לספק מוצר בסיסי גם בלעדיהן:

מטרה משנית ראשונה: ממשק גרפי מקצועי ומעוצב. ממשק משתמש אסתטי, עם ערכת צבעים עקבית, אנימציות עדינות, מחוונים ויזואליים ברורים, וחוויית שימוש נוחה. הממשק שבניתי מבוסס על ערכת צבעים כחולה-כהה עם אקסנטים ציאן, ומשתמש בגופן monospace שמוסיף תחושת מקצועיות טכנולוגית.

מטרה משנית שנייה: מסך פתיחה מרשים. שרת שמציג וידאו ואודיו לפני עלייתו לאוויר הוא פרט קטן אך משמעותי — הוא מייצר רושם ראשוני חיובי ומקצועי, ומבחין את המוצר מפתרונות "גנריים".

מטרה משנית שלישית: מצב אוטומטי לבדיקות. היכולת להריץ לקוחות ללא ממשק גרפי, עם קריאת פרטים מקבצים סטטיים, היא חיונית לביצוע בדיקות עומס. ללא מצב זה, הרצת 20 לקוחות בו-זמנית הייתה דורשת פתיחת 20 חלונות GUI בו-זמנית — דבר שאינו מעשי.

מטרה משנית רביעית: הגנת DDoS. אמנם זהו גם דרישה ביטחונית, אך הוספתי אותה לקטגוריה זו כי המערכת הייתה פועלת גם בלעדיה בתרחישי שימוש רגילים. ההגנה הופכת הכרחית רק כאשר יש ניסיון זדוני לפגוע בשרת.

1.3.3 מדדי הצלחה

כדי לדעת האם המטרות הושגו, הגדרתי מדדי הצלחה קונקרטיים וניתנים למדידה:

הושג?	ערך יעד	מדד
✓ 10fps על LAN תקין	≤ 10 פריימים/שנייה	קצב שידור מסך
✓ ~0.8 שנייה	< 1 שנייה	השהיה ניטור מקלדת
✓ נבדק ב-multi_run	15 ללא קריסה	מספר לקוחות בו-זמנית
✓ מיידי	< 3 שניות	זמן גילוי DDoS
✓ finally-ב cleanup	< 5 שניות	זמן תגובה לניתוק לקוח
✓ כל הקשה נשמרת	100% מהנתונים	שמירת keylog לקובץ
✓ AES-GCM על הכל	100% מהחבילות	הצפנת כל התקשורת
✓ multi_run.py עובד	ללא crashes	הרצת 20 threads

1.4 בעיות, תועלות וחסכונות

1.4.1 הגדרת הבעיה

הבעיה המרכזית שהפרויקט בא לפתור ניתנת לניסוח בצורה פשוטה: הורים מודרניים אינם יודעים מה ילדיהם עושים במחשב, ואין להם כלי נגיש, זמין ומאובטח שיאפשר להם לדעת זאת ברמה פרקטית.

כדי להבין את עומקה של הבעיה, יש לבחון אותה ממספר זוויות:

הזווית הראשונה היא הזמינות: ילדים בישראל ובעולם מבלים בממוצע 4–6 שעות ביום מול מסך. לרוב ההורים אין מושג מה מתרחש בשעות הרבות הללו. האם הילד מכין שיעורים? גולש ברשתות חברתיות? משחק משחקי מחשב? צופה בתכנים לא מתאימים? כל אחת מהשאלות הללו היא לגיטימית, ולכולן אין תשובה פשוטה ונגישה עבור ההורה הממוצע.

הזווית השנייה היא ההשלכות: שימוש לא מבוקר במחשב עלול להוביל להשלכות חמורות. הפרות בריאות הנפש הנובעות מחשיפה לתכנים פוגעניים, קשרים מסוכנים עם גורמים זרים ברשת, ירידה בהישגים לימודיים, הפרעות שינה, ובמקרים קיצוניים — מעורבות בפשיעה קיברנטית. מחקרים מצביעים על כך שגילוי מוקדם ופיקוח הוריים הם מגינים משמעותיים מפני סיכונים אלו.

הזווית השלישית היא הפער הטכנולוגי: פרדוקס מעניין הוא שדווקא ילדים בני 14–17 לרוב בקיאים טכנולוגית יותר מהוריהם. הורה שמנסה להתמודד עם הבעיה בכוחות עצמו — למשל, לבדוק היסטוריית גלישה — עלול לגלות שהילד מחק אותה, השתמש במצב גלישה פרטית, או השתמש ב-VPN. לכן, פתרון אמיתי לבעיה חייב לפעול ברמה עמוקה יותר — ברמת מערכת ההפעלה — ולא ברמת הגדרות הדפדפן.

1.4.2 ניתוח מה אנחנו מנסים להשיג

המערכת שפותחה אינה מנסה לפתור את כל בעיות הפיקוח ההורי — אין דבר כזה. היא מנסה לספק מענה ספציפי לצורך ספציפי: מידע בזמן אמת על פעילות המחשב של הילד, בצורה מאובטחת, אמינה ונגישה להורה.

אנחנו מנסים להשיג שקיפות מבלי לפגוע בפרטיות מוחלטת: ההורה מקבל חלון לפעילות הדיגיטלית של ילדו, אך הנתונים נשארים בתוך הבית ולא עוברים דרך שרתים חיצוניים. זהו שיקול חשוב — בעולם שבו גופים מסחריים מנצלים נתוני ילדים, מערכת שפועלת לחלוטין בתוך הרשת הביתית היא עדיפה מבחינת פרטיות.

אנחנו מנסים להשיג נגישות לכל הורה: הממשק תוכנן לאנשים שאינם טכנולוגיים. כפתורים גדולים, הודעות ברורות, אין צורך בהגדרות מורכבות. המטרה היא שהורה ממוצע יוכל להתחיל לנטר תוך דקות ספורות מרגע ההתקנה.

אנחנו מנסים להשיג אמינות לאורך זמן: מערכת שקורסת כל שעה היא גרועה מהיעדר מערכת, כי היא יוצרת אשליה של ביטחון. לכן, השקעתי מאמץ רב בטיפול בכל המקרים הקצה — ניתוקים פתאומיים, הודעות לא צפויות, כשלי DB — כדי שהמערכת תפעל בצורה יציבה ואמינה.

1.4.3 תועלות מרכזיות של המערכת

התועלת הראשונה היא הגנה פרואקטיבית: ההורה אינו צריך להמתין לכשל כדי לגלות שמשהו השתבש. הוא יכול לצפות בזמן אמת ולהתערב מיד אם הוא מזהה פעילות בעייתית. ניטור פרואקטיבי מוכח כמפחית בצורה משמעותית את הסיכון לפגיעה בילדים.

התועלת השנייה היא תיעוד לצורך עדות: יומני המקלדת הנשמרים בבסיס הנתונים ובקבצים מספקים תיעוד כתוב של הפעילות. במקרים קיצוניים שבהם נדרש תיעוד לצורך הליכים משפטיים או חינוכיים, הנתונים זמינים ומאורגנים.

התועלת השלישית היא הרתעה: לעתים קרובות, עצם הידיעה של הילד שפעילותו עשויה להיות מנוטרת (גם אם לא בכל רגע) מפחיתה את הנטייה להתנהגויות בעייתיות. הרתעה מניעתית היא כלי יעיל.

התועלת הרביעית היא חיסכון כלכלי: הפתרונות המסחריים הקיימים עולים בין 50 ל-150 דולר לשנה. פתרון קוד פתוח שפועל ללא עלות ריצה מציע חיסכון משמעותי לאורך שנים.

התועלת החמישית היא עצמאות מספקים חיצוניים: בניגוד לפתרונות ענן, המערכת הזו אינה תלויה בכך שחברה מסחרית תמשיך לפעול, לא תשנה את תנאי השירות, ולא תעלה מחירים. אחרי ההתקנה הראשונית, המערכת פועלת לחלוטין באופן עצמאי.

1.4.4 חסכונות ויתרונות תפעוליים

מעבר לתועלות הישירות, יש לציין מספר יתרונות תפעוליים שמייחדים את הפתרון הזה לעומת הפתרונות המסחריים:

חיסכון בעלויות: פתרון המרכזי מסוג Qustodio עולה כ-180 שקל לחודש. לאורך שנה — 2,160 שקל. מערכת שפותחה פעם אחת ופועלת ללא עלות שוטפת מציעה חיסכון ממשי.

חיסכון בפרטיות: שליחת נתוני ניטור — ובמיוחד תמונות מסך והקשות מקלדת של ילד — לשרתי ענן של חברה מסחרית זרה היא סוגיה אתית ופרטיותית מורכבת. המערכת המקומית מבטיחה שהנתונים לא עוזבים את הבית.

חיסכון בסמכות: הורה שמשתמש בפתרון מסחרי תלוי בהחלטות של החברה. אם החברה תחליט לשנות את הממשק, להסיר פיצ'ר, או לסגור את השירות — ההורה מאבד גישה. עם פתרון עצמאי, הורה שואל את עצמו מי שולט במה שקורה במחשבי ביתו — והתשובה צריכה להיות הוא עצמו.

1.5 סקירת פתרונות קיימים

1.5.1 מבוא לסקירה

לפני כל פרויקט תוכנה רציני, על המפתח לבצע סקירה מקיפה של הפתרונות הקיימים בשוק. סקירה זו משרתת מספר מטרות: היא מסייעת להבין מה כבר נעשה ואין טעם לחדש את הגלגל; היא עוזרת לזהות פערים שהפתרונות הקיימים אינם ממלאים; ומחדדת את ההבנה של מה שהמוצר שלנו צריך להיות שונה וטוב יותר. להלן סקירה מפורטת של הפתרונות העיקריים הקיימים כיום בשוק.

Qustodio 1.5.2

Qustodio היא אחת מתוכנות בקרת ההורים הנפוצות והמוכרות ביותר בעולם. פותחה בספרד ומשווקת לשוקי אמריקה, אירופה וישראל, החברה מספקת שירות מנוי שמציע ניטור פעיל של מכשירים ביתיים.

היכולות של Qustodio כוללות: סינון אתרים לפי קטגוריות (פורנוגרפיה, אלימות, הימורים, רשתות חברתיות), ניהול זמן מסך ומגבלות שימוש, דוחות שבועיים ויומיים על פעילות הגלישה, חסימת אפליקציות, מעקב מיקום GPS במכשירים ניידים, וניטור רשתות חברתיות מסוימות. יתרונות: ממשק ידידותי ונוח לשימוש, תמיכה בפלטפורמות מרובות (Windows, Mac, Android, iOS), ותמיכה טכנית טובה.

חסרונות קריטיים: הפתרון עולה בין 54.95 דולר ל-137.95 דולר לשנה — עלות שנהייה מצטברת לסכום משמעותי לאורך שנים; כל הנתונים עוברים דרך שרתי Qustodio בספרד — מה שמעלה חשש לפרטיות; אין יכולת שידור מסך בזמן אמת; אין ניטור הקשות מקלדת; ולמרות הטענה לניטור רשתות חברתיות, היכולת הזו מוגבלת ותלויה בפלטפורמה.

Net Nanny 1.5.3

Net Nanny, שפותחה בקנדה ונרכשה על ידי חברת ContentWatch, היא אחת התוכנות הוותיקות בתחום — פועלת מאז 1994. היא התמקדה מסורתית בסינון תכנים ובמניעת חשיפה לא הולמת.

היכולות של Net Nanny כוללות: סינון אתרים מתוחכם עם עדכוני מסד נתונים יומיים, חסימת אפליקציות, ניהול זמן מסך, אזהרות בזמן אמת כאשר ילד מנסה לגשת לתוכן חסום, ודוחות פעילות. יתרונות: ניסיון רב שנים בתחום, מסד נתונים גדול של אתרים מסוננים, וממשק פשוט.

חסרונות קריטיים: גם Net Nanny דורשת מנוי שנתי (39.99–89.99 דולר); אין ניטור בזמן אמת — רק דוחות רטרוספקטיביים; אין שידור מסך; אין ניטור מקלדת; והמוצר ידוע כניתן לעקיפה יחסית קלה על ידי בני נוער טכנולוגיים.

Google Family Link 1.5.4

Family Link הוא שירות חינמי של Google, שהושק בשנת 2017 ומיועד בעיקר לניהול מכשירים ניידים של ילדים. הוא מיועד בעיקר להורים שרוצים לנהל את מכשיר ה-Android של ילדם.

היכולות של Family Link כוללות: ניהול אפליקציות (אישור הורשות לפני הורדה), מגבלות זמן מסך, מעקב מיקום, ניהול הגדרות Google Account של הילד, ודוחות שימוש שבועיים. יתרונות: חינמי לחלוטין, אינטגרציה מצוינת עם מכשירי Android ושירותי Google.

חסרונות קריטיים: מוגבל כמעט לחלוטין למכשירים ניידים עם Android — תמיכה מוגבלת מאוד ב-Windows/Mac/iOS; אין שידור מסך; אין ניטור מקלדת; תלוי לחלוטין בחשבון Google ובמדיניות Google שמשתנה מעת לעת; ואינו מתאים לסביבות שבהן הילד משתמש במחשב שולחני או נייד.

Microsoft Family Safety 1.5.5

Microsoft Family Safety הוא שירות חינמי/פרמיום של מיקרוסופט המשולב ב-Windows. הוא חלק ממנוי Microsoft 365 Family.

היכולות כוללות: סינון אתרים ב-Microsoft Edge, ניהול זמן מסך, דוחות פעילות, ניהול רכישות ב-Microsoft Store, ומעקב מיקום. יתרונות: שילוב טבעי עם Windows, ממשק נוח, ללא עלות נוספת לבעלי Microsoft 365.

חסרונות קריטיים: הסינון עובד רק ב-Edge — שימוש בדפדפן אחר עוקף את ההגנה לחלוטין; אין שידור מסך; אין ניטור מקלדת; הדוחות הם יומיים/שבועיים ולא בזמן אמת; ואינו פועל טוב בסביבות שבהן הילד הוא Admin על המחשב.

1.5.6 מסקנות הסקירה והמיצוב של הפרויקט

לאחר סקירה מקיפה של הפתרונות הקיימים, עולה תמונה ברורה: אין כיום פתרון בשוק שמשלב שידור מסך בזמן אמת עם ניטור מקלדת, פועל לחלוטין ברשת מקומית ללא תלות בשרתי ענן, מוצפן, חינמי ובעל ממשק גרפי נוח — כל זאת יחד.

יכולת	Qustodio	Net Nanny	Family Link	MS Family	הפרויקט שלנו
שידור מסך בזמן אמת	✗	✗	✗	✗	✓
ניטור מקלדת	✗	✗	✗	✗	✓
הצפנת תקשורת	✓	✓	✓	✓	✓
ללא מנוי חודשי	✗	✗	✓	✓	✓
פועל ללא שרת ענן	✗	✗	✗	✗	✓
הגנת DDoS	✗	✗	✗	✗	✓
תמיכה ב-Windows	✓	✓	✗	✓	✓
קוד פתוח מותאם אישית	✗	✗	✗	✗	✓

כפי שניתן לראות בטבלה לעיל, הפרויקט שפותח מציע יכולות ייחודיות שאין בשום פתרון מסחרי קיים: שידור מסך בזמן אמת, ניטור מקלדת, פעולה מקומית מלאה, והגנת DDoS מובנית. שילוב יכולות אלו יחד עם אפס עלות שוטפת הופך אותו לפתרון שמשלים פער אמיתי בשוק.

1.6 סקירה טכנולוגית

1.6.1 שפת הפיתוח: Python 3.8

Python היא שפת תכנות פרשנית ברמה גבוהה, שנוצרה על ידי Guido van Rossum ושוחררה לראשונה בשנת 1991. כיום, Python היא אחת משלוש שפות התכנות הפופולריות ביותר בעולם, ומשמשת בתחומים מגוונים מאוד: פיתוח ווב, מדע נתונים, בינה מלאכותית, אוטומציה, ופיתוח אפליקציות.

הסיבות לבחירה ב-Python לפרויקט זה הן מספר: ראשית, Python מספקת תמיכה מצוינת ב-socket programming — ספריית socket המובנית מאפשרת יצירת חיבורי TCP/IP בכמה שורות קוד, מבלי צורך בניהול ידני של זיכרון או מבני נתונים ברמה נמוכה. שנית, לספריית threading של Python יש תמיכה מלאה ב-locks, daemon threads, ו-events שנדרשים לניהול חיבורים מקבילים. שלישית, האקוסיסטם של Python מציע ספריות איכותיות לכל צורך — מהצפנה (PyCryptodome) ועד עיבוד תמונה (Pillow, OpenCV). רביעית, הקוד של Python קריא ומובן, מה שמקל על תחזוקה ותיעוד.

הגרסה הנבחרת היא 3.8, שהייתה הגרסה היציבה ברוב תקופת הפיתוח. גרסה זו תומכת ב-f-strings, ב-walrus operator, ובמספר שיפורים ביצועיים שהקלו על הפיתוח.

1.6.2 תקשורת רשת: TCP/IP ו-Sockets

תקשורת ה-TCP/IP (Transmission Control Protocol / Internet Protocol) היא הבסיס של כל תקשורת רשת מודרנית. TCP מספק חיבור אמין ומסודר בין שני קצוות — כל חבילת נתונים מגיעה בסדר הנכון, ואם חבילה אובדת בדרך, היא נשלחת מחדש אוטומטית. זהו הפרוטוקול המתאים ביותר לפרויקט זה כי:

ראשית, אמינות: כאשר שולחים נתוני מקלדת, חשוב שכל הקשה תגיע — לא ניתן להרשות לעצמי אובדן נתונים. שנית, סדר: הודעות חייבות להגיע בסדר שנשלחו כדי שהמסרים יתפרשו נכון בצד המקבל. שלישית, זרימה: TCP מנהל בעצמו את בקרת הזרימה ואת החלוניות, מה שמפשט מאוד את הקוד.

ה-socket הוא הממשק התכנותי לתקשורת TCP. כל חיבור לקוח-שרת מנוהל דרך זוג סוקטים — אחד בכל קצה. הנתונים שנשלחים דרך הסוקט הם רצף bytes גולמי, ולכן כל הודעה מוקדמת ב-4 bytes המציינים את אורכה — כך הצד המקבל יודע בדיוק כמה bytes לקרוא.

1.6.3 ספריית Tkinter לממשק גרפי

Tkinter (ראשי תיבות של Tk interface) היא ספריית ממשק משתמש גרפי המובנית בתוך Python ומגיעה עם כל התקנה סטנדרטית. היא עוטפת את ספריית Tcl/Tk שפותחה בשנות התשעים ועדיין מתוחזקת באופן פעיל.

Tkinter מספקת רכיבי UI בסיסיים: Label (תוויות טקסט), Entry (שדות קלט), Button (כפתורים), Canvas (ציור חופשי), Text (טקסט ארוך עם גלילה), Frame (מכלים לרכיבים), וRadiobutton (לחצני בחירה). ניתן לעצב כל רכיב עם צבעי רקע ומסגרת מותאמים אישית, ולבנות ממשק מסודר ואסתטי.

האתגר המרכזי שנתקלתי בו עם Tkinter הוא מגבלת ה-thread-safety שלה. בניגוד לספריות UI מודרניות כמו Qt שמספקות מנגנוני תקשורת בין-threads, Tkinter מחייבת שכל עדכוני ה-UI יבוצעו מ-thread אחד בלבד — ה-main thread. כאשר thread רקע (כמו ה-thread שמקבל נתוני שידור מסך) מנסה לעדכן ישירות רכיב UI, הקוד עלול להיכנס ל-deadlock, לקרוס בשקט, או לייצר תוצאות לא צפויות. הפתרון הסטנדרטי הוא שימוש בפונקציה root.after(0, function, args) שמוסיפה קריאת פונקציה לתור האירועים של Tkinter ומבטיחה שהיא תבוצע ב-main thread.

1.6.4 הצפנת AES-GCM

AES (Advanced Encryption Standard) הוא אלגוריתם הצפנה סימטרי שנקבע כתקן על ידי ה-NIST (National Institute of Standards and Technology) בשנת 2001. כיום, AES נחשב לאלגוריתם ההצפנה הסימטרי הבטוח ביותר הקיים לשימוש מסחרי ומשמש כבסיס לכמעט כל פרוטוקול תקשורת מאובטח בעולם — כולל HTTPS, WPA2, ו-TLS.

GCM (Galois/Counter Mode) הוא מצב פעולה של AES שמוסיף שכבת authentication מעל ההצפנה. בניגוד ל-CBC (Cipher Block Chaining) שמספק רק סודיות, GCM מספק גם integrity — כלומר, הצד המקבל יכול לוודא שההודעה לא שונתה בדרך. ב-GCM, כל הודעה מוצפנת מלווה ב-authentication tag בגודל 16 bytes. אם גורם שלישי ישנה אפילו bit אחד ב-ciphertext, בדיקת ה-tag תיכשל והמסר יידחה.

בפרויקט, אורך המפתח הוא 256 bits (32 bytes) — הגרסה החזקה ביותר של AES. ה-Nonce (Number used once) הוא 12 bytes כפי שמומלץ על ידי ה-NIST עבור GCM. ה-tag בגודל 16 bytes מצורף ל-ciphertext לפני שליחה. הספרייה PyCryptodome מספקת מימוש יעיל ובדוק של AES-GCM.

1.6.5 ספריית PyAutoGUI לצילום מסך

PyAutoGUI היא ספריית Python רב-תכליתית המאפשרת שליטה תכנותית בממשק המשתמש של מערכת ההפעלה: הזזת עכבר, לחיצות על כפתורים, הקלדת טקסט, וצילום מסך. בפרויקט זה נעשה שימוש יחודי ביכולת צילום המסך.

הפונקציה pyautogui.screenshot() מחזירה אובייקט PIL.Image המכיל צילום של המסך הנוכחי ברזולוציה המלאה. בפרויקט, הצילום עובר מיד דחיסה לרזולוציה 480×270 (רבע ה-Full HD) ושמירה כ-JPEG באיכות 50%. בחירות אלו היו מכוונות: רזולוציה מוקטנת מפחיתה את גודל הקובץ פי 4, ואיכות JPEG 50% מפחיתה את הגודל עוד יותר מבלי שהתמונה תהפוך ללא ניתנת

לקריאה. התוצאה הסופית היא פריים בגודל של כ-15–30 קילובייט, שניתן לשלוח על רשת מקומית תוך אלפיות שנייה.

1.6.6 ספריית pynput לניטור מקלדת

pynput היא ספריית Python המספקת האזנה לאירועי עכבר ומקלדת ברמת מערכת ההפעלה. בניגוד לקריאת קלט מהמסוף, pynput מאזינה לכל אירועי המקלדת בכל יישום פעיל — זה מה שמבדיל keylogger אמיתי מקריאת קלט פשוטה.

ה-Listener של pynput פועל ב-thread נפרד לחלוטין מה-main thread ומה-thread התקשורת. הוא ממתין לאירועי מקלדת ומפעיל on_press) callback function) בכל לחיצת מקש. בפרויקט, ה-callback מוסיף את הצ'אר לרשימה (key_buffer) ובודק האם כדאי לשלוח עוד נתונים — לפי גודל החבילה (20 תווים) או לפי זמן (0.8 שנייה).

1.6.7 בסיס הנתונים: MySQL / MariaDB

MySQL היא מערכת ניהול בסיסי נתונים רלציונית (RDBMS) קוד-פתוח שפותחה ב-1995. MariaDB היא fork של MySQL שנוצרה ב-2009 ומשמשת כתחליף תואם לחלוטין. בפרויקט נעשה שימוש ב-MariaDB דרך חבילת XAMPP.

בסיס הנתונים משמש לשלוש מטרות עיקריות בפרויקט: ראשית, אחסון פרטי המשתמשים — שמות משתמש, סיסמאות מוצפנות, תפקידים, ומידע על הצמדת הורה-ילד. שנית, תיעוד יומני מקלדת — שמירת הנתיב לכל קובץ יומן שנשמר, עם מזהה ההורה והילד המשויכים. שלישית, ניהול מצב DDoS — שמירת סטטוס חסימה לכל משתמש.

הגישה ל-MySQL נעשית דרך ספריית mysql-connector-python שמספקת ממשק Python ידידותי. כל פעולות ה-SQL עטופות במחלקת DatabaseManager שמספקת ממשק אחיד ומחסנת את כל פרטי החיבור.

1.6.8 ספריות מולטימדיה: OpenCV ו-Pygame

OpenCV (Open Source Computer Vision Library) היא ספריית עיבוד תמונה ועיבוד וידאו עשירה מאוד. בפרויקט, השימוש בה מוגבל לקריאת קובץ ה-splash.mp4 frame-by-frame ולהמרת כל פריים מפורמט BGR (שנהוג ב-OpenCV) לפורמט RGB (שנהוג ב-Pillow).

Pygame היא ספריית Python לפיתוח משחקים ואפליקציות מולטימדיה. בפרויקט, היא משמשת אך ורק לניגון קובץ האודיו splash_audio.mp3 במקביל לשידור הווידאו. Pygame.mixer מספקת ניגון אסינכרוני שמאפשר לווידאו להמשיך לרוץ בלי להמתין לסיום ניגון האודיו.

1.6.9 סייגים ומגבלות הטכנולוגיות

מגבלה ראשונה — מגבלת מערכת הפעלה: הפרויקט פועל על Windows בלבד. ספריית pyautogui צולמת מסך תומכת גם ב-Linux ו-macOS, אולם ספריית pynput עשויה לדרוש הרשאות מיוחדות ב-X11) Linux) ואין לה תמיכה מלאה ב-Wayland. לכן, ב-scope הפרויקט הנוכחי, Windows היא הפלטפורמה הנתמכת הרשמית.

מגבלה שנייה — תלות ברשת מקומית: המערכת מחייבת שכל המשתמשים יהיו באותה רשת מקומית. אין תמיכה בניטור מרחוק דרך אינטרנט. זה למעשה יתרון מבחינת אבטחה ופרטיות, אבל מגביל את הגמישות.

מגבלה שלישית — מפתח הצפנה קבוע: בגרסה הנוכחית, מפתח ה-AES הוא קבוע ומוטמע בקוד. בסביבת ייצור אמיתית, היה צריך להשתמש במנגנון Diffie-Hellman להחלפת מפתחות דינמיים בתחילת כל סשן. זהו שיפור ידוע שמתוכנן לגרסה עתידית.

מגבלה רביעית — הרשאות מנהל: ספריית pynput לניטור מקלדת מחייבת הרשאות מנהל (Administrator) ב-Windows כדי לנטר הקשות בכל היישומים. ללא הרשאות אלו, הניטור יתבצע רק בתוך החלון הפעיל של הלקוח.

1.7 תיחום הפרויקט

1.7.1 תחומים בהם הפרויקט עוסק

תיחום הפרויקט הוא ההגדרה הברורה של מה שנמצא בתחום הפרויקט ומה שנמצא מחוץ לו. הגדרה זו היא קריטית לניהול ציפיות ולהגדרת הצלחה. להלן פירוט של התחומים המרכזיים שהפרויקט עוסק בהם:

תחום ראשון — תקשורת רשת מקומית: הפרויקט עוסק באופן מלא בתקשורת TCP/IP בסביבת LAN. זה כולל ניהול חיבורים מרובים, טיפול בניתוקים ומנגנוני reconnect, פרוטוקול תקשורת מוגדר היטב עם מבנה הודעות ברור, וסנכרון בין threads מרובים שמנהלים חיבורים שונים.

תחום שני — אבטחת סייבר: הפרויקט מיישם מספר עקרונות אבטחה מרכזיים שנלמדו בחלופת הסייבר. ההצפנה מבוססת AES-GCM מגנה על סודיות ושלמות התקשורת. מנגנון hash סיסמאות ב-SHA-256 מגן מפני פריצת בסיס הנתונים. הגנת DDoS מגנה על זמינות השרת. ובקרת גישה מבוססת תפקידים (RBAC) מבטיחה שכל משתמש יכול לגשת רק לפונקציות המתאימות לתפקידו.

תחום שלישי — בסיסי נתונים: הפרויקט כולל עיצוב, מימוש ותחזוקה של בסיס נתונים רלציוני. זה כולל תכנון סכמת טבלאות עם קשרי גומלין ברורים, מניעת SQL Injection דרך שימוש ב-parameterized queries, ניהול transactions, וחיבור בין הקוד לבסיס הנתונים דרך מחלקה מסכמת.

תחום רביעי — פיתוח ממשק משתמש: עיצוב ומימוש ממשק גרפי מלא עם Tkinter, כולל ניהול מצבי מסך שונים, אנימציות, מחוונים ויזואליים, ועיצוב אסתטי מקצועי. הפרויקט עוסק גם בבעיות thread-safety בממשק UI — נושא מורכב שנפוץ בפיתוח יישומים מרובי-threads.

תחום חמישי — עיבוד מדיה: צילום מסך, דחיסת תמונה, קידוד/פענוח Base64, קריאת וידאו frame-by-frame, וניגון אודיו — כל אלו נדרשו לצורך מימוש שידור המסך ומסך הפתיחה.

1.7.2 גבולות הפרויקט — מה המערכת אינה מטפלת

הגדרת מה שמחוץ ל-scope חשובה לא פחות מהגדרת מה שבתוכו. להלן הגבולות הברורים:

אין חסימת אתרים: המערכת היא מערכת ניטור — היא מספקת מידע להורה אך אינה חוסמת גישה לאתרים, אפליקציות, או תכנים. חסימה דורשת שינוי הגדרות DNS, Firewall, או proxy, שהם מחוץ ל-scope הפרויקט הנוכחי.

אין ניטור דרך אינטרנט: הפרויקט מותחם לרשת מקומית בלבד. ניטור מרחוק דרך WAN/אינטרנט דורש תצורת NAT, tunneling, ואבטחה מחמירה יותר — שאף הם מחוץ ל-scope.

אין תמיכה בניידים: מכשירי iOS ו-Android לא נתמכים. הפרויקט מוגבל למחשבים שולחניים/ניידים עם Windows.

אין ניטור שמע: ניטור מיקרופון אינו מיושם בגרסה הנוכחית, למרות שקיים שדה client_total_audio_logs בבסיס הנתונים שמאפשר הרחבה עתידית.

אין ממשק וב: כל הגישה למערכת היא דרך יישום Desktop בלבד. אין ממשק דפדפן.

אין הקלטת וידאו: המערכת מציגה שידור מסך בזמן אמת אך אינה שומרת הקלטת וידאו של הפעילות לאורך זמן.

1.8 אפיון המערכת

1.8.1 תיאור מפורט של המערכת

מערכת בקרת ההורים היא יישום Python מורכב המורכב ממספר רכיבים הפועלים יחד. ארכיטקטורת הבסיס היא Client-Server קלאסית: שרת מרכזי אחד פועל על מחשב ההורה ומנהל את כל ההיגיון; לקוחות (clients) מתחברים אליו מהמחשבים השונים ומתקשרים עמו.

פרוטוקול הפעולה הוא כדלקמן: בעת הפעלת השרת, מסך הפתיחה מוצג (splash.py) ולאחר מכן השרת מתחיל להאזין לחיבורים נכנסים על port 9921. כאשר לקוח מתחבר, השרת בודק תחילה אם מגבלות ה-DDoS מופרות — אם כן, הוא מנתק את הלקוח מיד. אם לא, הוא פותח thread ייעודי לטיפול בלקוח זה ומחכה לפקודות.

הלקוח, בעת הפעלתו, בודק תחילה אם קיים קובץ פרטי כניסה (creds_*.txt) — אם כן, הוא מתחבר בצורה אוטומטית ללא ממשק גרפי. אם לא — הממשק הגרפי נפתח ומציג מסך כניסה. לאחר כניסה מוצלחת, הממשק מתעדכן בהתאם לתפקיד: ממשק ניטור מלא להורה, ומסך מינימלי לילד.

כל התקשורת בין הלקוח לשרת מנוסחת בפרוטוקול pipe-delimited: כל הודעה היא מחרוזת טקסט שבה שדות מופרדים בתו |. לדוגמה: LOGIN|username|password. הפרוטוקול נועד להיות פשוט לפענוח ועמיד לטיפול בנתונים בינאריים (כמו תמונות המקודדות ב-Base64).

1.8.2 יכולות מפורטות לפי סוג משתמש

יכולות משתמש הורה — לאחר כניסה מוצלחת, ההורה מקבל גישה לממשק הניטור המלא הכולל:

- הפעלת שידור מסך: לחיצה על כפתור ▶ START STREAM שולחת את הפקודה FORWARD|SCREEN_START לשרת, שמעבירה אותה לסיבלינג (הילד). הילד מתחיל לשלוח פריימי מסך שמוצגים בקנבס של ההורה בזמן אמת.
- עצירת שידור מסך: לחיצה על ■ STOP STREAM שולחת FORWARD|SCREEN_STOP. השידור נעצר והקנבס מתנקה.
- הפעלת ניטור מקלדת: לחיצה על ⏺ START LOG שולחת FORWARD|KEYLOG_START. הילד מפעיל Listener. הקשות מגיעות לממשק ההורה עם חותמות זמן.
- עצירת ניטור מקלדת: לחיצה על ⏹ STOP LOG שולחת FORWARD|KEYLOG_STOP. השרת שומר את כל הנתונים שנאספו לקובץ txt בתיקיית /keylogs ומתעד בבסיס הנתונים.
- שעון חי: בפינה הימנית העליונה של הממשק מוצג שעון חי שמתעדכן כל שנייה — עוזר להורה לדעת מתי הוא מנטר.
- מחווני סטטוס: כל פאנל (שידור ומקלדת) מציג מחוון IDLE/LIVE שמתעדכן בזמן אמת.

יכולות משתמש ילד — לאחר כניסה מוצלחת, הילד רואה:

- מסך SESSION ACTIVE עם נקודה ירוקה מהבהבת
- שעון חי שמציג את השעה הנוכחית
- הודעה שמסבירה שניטור פועל ברקע
- אין גישה לפקדי ניטור, אין אפשרות לכבות מהממשק

1.8.3 לוח זמנים ראשוני מול לוח זמנים בפועל

בתחילת הפרויקט, הגדרתי לוח זמנים ראשוני לפיתוח. בפועל, חלק מהשלבים ארכו זמן רב יותר מהמתוכנן, בעיקר בשל בעיות טכניות שלא ציפיתי להן. הטבלה הבאה מציגה את ההשוואה:

הפרש	בוצע בפועל	תכנון מקורי	שלב פיתוח
0	ספטמבר 2024 (2 שבועות)	ספטמבר 2024 (2 שבועות)	ארכיטקטורה ותכנון ראשוני
0	אוקטובר 2024 (3 שבועות)	אוקטובר 2024 (3 שבועות)	תקשורת בסיסית שרת-לקוח
+1 שבוע	נובמבר 2024 (3 שבועות)	אוקטובר 2024 (שבועיים)	הצפנת תקשורת AES-GCM
0	נובמבר 2024 (3 שבועות)	נובמבר 2024 (3 שבועות)	ניהול משתמשים + DB
+2 שבועות	דצמבר 2024 (5 שבועות)	נובמבר 2024 (3 שבועות)	שידור מסך
0	דצמבר 2024 (שבועיים)	דצמבר 2024 (שבועיים)	ניטור מקלדת
+1 שבוע	ינואר 2025 (4 שבועות)	ינואר 2025 (3 שבועות)	GUI מקצועי ומעוצב
דחייה	פברואר 2025 (שבועיים)	ינואר 2025 (שבועיים)	הגנת DDoS
0	פברואר 2025 (שבועיים)	פברואר 2025 (שבועיים)	מצב headless + multi_run
+1 שבוע	מרץ 2025 (שבועיים)	פברואר 2025 (שבוע)	מסך פתיחה + אודיו
0	מרץ 2025 (3 שבועות)	מרץ 2025 (3 שבועות)	בדיקות, bug fixes, תיעוד

השלב שחרג הכי הרבה מהתכנון היה שידור המסך — חריגה של שבועיים. הסיבה העיקרית הייתה בעיית ה-thread-safety של Tkinter שתוארה בפרק הרפלקציה. שלב ההצפנה חרג בשבוע בגלל הצורך ללמוד AES-GCM מאפס. בסך הכל, חרגתי מהתכנון ב-5 שבועות — אך הגעתי למוצר סופי איכותי ועובד.

1.8.4 ניהול סיכוני הפרויקט

לפני תחילת הפיתוח, זיהיתי מספר סיכונים פוטנציאליים ותכננתי דרכי התמודדות. להלן ניתוח מפורט של כל סיכון:

סיכון ראשון: בעיות ביצועים בשידור מסך בזמן אמת. הערכת חומרה: גבוהה. ציפיתי שייתכן שהרשת המקומית לא תוכל לתמוך בקצב הפריימים הנדרש. תכנון: לנסות רזולוציות שונות ואיכויות

JPEG שונות עד למציאת האיזון הנכון. בפועל: הגעתי ל-480×270 JPEG 50% שנותן קצב 10fps מספק ללא עומס יתר על הרשת.

סיכון שני: בעיות thread-safety. הערכת חומרה: בינונית בתחילה, גבוהה בפועל. לא העריכתי מספיק את מורכבות ניהול threads עם Tkinter. בפועל: השקעתי שבוע שלם בפתרון הבעיה דרך שימוש ב-root.after(0, ...) לכל עדכוני UI.

סיכון שלישי: כשל בסיס נתונים. הערכת חומרה: נמוכה. השרת יכול לקרוס אם MySQL לא זמין. בפועל: הוספתי try/except לכל פעולות DB, ובנוסף create_all_tables נקרא עם בדיקה אם הטבלאות קיימות כבר.

סיכון רביעי: ניסיון עקיפה על ידי ילד. הערכת חומרה: בינונית. ילד טכנולוגי עשוי לנסות לסגור את תהליך הלקוח. בפועל: טיפלתי בכך בהבנה שמניעה מוחלטת מחוץ לscope, אך הפכתי את ממשק הלקוח לנסתר ככל הניתן.

סיכון חמישי: עלות ביצועי מעבד. הערכת חומרה: בינונית. ניטור מסך בקצב גבוה עלול לאיטי את מחשב הילד. בפועל: המדידות הראו שימוש ב-5–15% מעבד — ניתן לשיפור בגרסאות עתידיות.

1.9 סיכום פרק המבוא

פרק זה הציג את הבסיס לכל מה שיבוא בפרקים הבאים. סקרנו את הרציונל לפיתוח מערכת בקרת הורים מקומית, את קהל היעד ואת צרכיו, את היעדים שהוגדרו ואת מדדי ההצלחה, את הבעיות שהמערכת באה לפתור ואת התועלות שהיא מספקת, את הפתרונות המסחריים הקיימים ואת הפרמטרים שמבחינים את הפרויקט הזה מהם, את הטכנולוגיות שנבחרו ומדוע, ואת תיחום הפרויקט המדויק.

הנקודה המרכזית שעולה מפרק זה היא שהפרויקט אינו עוד יישום ניטור גנרי — הוא פתרון ממוקד שמתייחס לצרכים ספציפיים שאין להם מענה בשוק כיום: שידור מסך בזמן אמת, ניטור מקלדת, פעולה מקומית מלאה ללא שרתי ענן, הצפנה, והגנת DDoS — כל זאת יחד, חינם, עם ממשק גרפי מקצועי.

הפרקים הבאים יצללו לעומק לתוך הארכיטקטורה הטכנית של המערכת, מימוש הרכיבים השונים, ותוצאות הבדיקות שבוצעו.

פרק 2: מבנה וארכיטקטורה

2.1 ארכיטקטורת המערכת

המערכת בנויה על ארכיטקטורת Client-Server קלאסית עם שכבת הצפנה מלאה בין כל הרכיבים. השרת הוא הגורם המרכזי המנהל את כל ההיגיון העסקי, בעוד הלקוחות הם אפליקציות קלות יחסית.

רכיבי המערכת

רכיב	תיאור ותפקיד
`server.py`	השרת המרכזי. מנהל חיבורים, מעביר מסרים, מתנהל מול DB, מנהל הגנת DDoS.
`client.py`	הלקוח. מספק ממשק גרפי להורה ולילד, שולח שידור מסך ונתוני מקלדת.
`splash.py`	מסך פתיחה גרפי עם וידאו ואודיו, פועל לפני הפעלת השרת.
`encrypt.py`	מחלקת הצפנה/פענוח AES-GCM. משמשת שרת ולקוח כאחד.
`db_manager.py`	מחלקת ניהול בסיס הנתונים. עוטפת את כל פעולות ה-SQL.
`constants.py`	קובץ הגדרות מרכזי: IP, PORT, מגבלות חיבורים, פרמטרי DB.
`multi_run.py`	סקריפט הפעלה מרובת-לקוחות: 20 threads בו-זמנית.
`create_tables.py`	יוצר את טבלאות DB בהרצה הראשונה.
`creds_1-10.txt`	קבצי פרטי כניסה סטטיים לשימוש ב-multi_run.

דיאגרמת ארכיטקטורה

להלן תיאור מילולי של קשרי הרכיבים (ניתן לראות את הדיאגרמה המלאה בנספחים):

- מחשב ההורה: מריץ את server.py ואת client.py (תפקיד הורה)
- מחשב הילד: מריץ את client.py (תפקיד ילד) ומתחבר לשרת
- שרת ← → לקוח הורה: העברת פקודות ניטור ונתוני תצוגה
- שרת ← → לקוח ילד: העברת פקודות ניטור ונתוני שידור
- שרת ← → MySQL: שמירה וקריאת נתוני משתמשים ויומנים
- כל תקשורת: מוצפנת AES-GCM, TCP/IP, port 9921

2.2 טכנולוגיות ושפות תכנות

טכנולוגיה	גרסה	שימוש בפרויקט
Python	3.8	שפת פיתוח ראשית
Tkinter	מובנה	ממשק משתמש גרפי
PyCryptodome	x.3	הצפנת AES-GCM
PyAutoGUI	x.0.9	צילום מסך
pynput	x.1.7	האזנה למקלדת
mysql-connector	x.8	חיבור ל-MySQL
OpenCV	x.4	קריאת וידאו לספלאש
Pygame	2.6	ניגון אודיו
Pillow	x.10	עיבוד תמונה
MySQL/MariaDB	x.8	בסיס נתונים
threading	מובנה	ניהול threads מקביליים
socket	מובנה	תקשורת TCP/IP

סביבת הפיתוח

- עורך קוד: Visual Studio Code עם תוספי Python
- מערכת הפעלה לפיתוח: Windows 11
- כלי ניהול חבילות: pip
- בסיס נתונים: XAMPP עם MariaDB
- בדיקות רשת: שני מחשבים פיזיים באותה רשת Wi-Fi

2.3 זרימת מידע במערכת

זרימת הרשמה

1. הלקוח שולח: REGISTER|username|password|role (ובמידת הצורך |parent_id)
2. השרת בודק: האם המשתמש קיים? האם ה-parent_id תקין?
3. השרת מחשב hash לסיסמה ומכניס שורה חדשה לטבלת clients
4. אם ילד – השרת מעדכן את sibling_id אצל שניהם
5. השרת שולח: REGISTER_OK|new_id

זרימת כניסה

1. הלקוח שולח: LOGIN|username|password
2. השרת מחשב hash לסיסמה ומשווה מול DB
3. השרת בודק ddos_status – אם true, שולח LOGIN_FAIL|DDOS_BLOCKED
4. אם תקין – השרת שולח LOGIN_OK|role|sibling_status
5. השרת מוסיף את ה-socket למילון clients ומעדכן ip בDB

זרימת שידור מסך

1. ההורה לוחץ START STREAM → הלקוח שולח: FORWARD|SCREEN_START
2. השרת מעביר את ההודעה לסיבלינג (הילד)
3. הילד מתחיל לצלם מסך בקצב 10fps, דוחס ל-JPEG 50%, מקודד Base64
4. הילד שולח: <FORWARD|SCREEN_DATA|<base64_data
5. השרת מעביר לסיבלינג (ההורה)
6. הלקוח ההורה מקבל, מפענח Base64, פותח תמונה ומציג בקנבס
7. כל עדכון UI מתוזמן דרך root.after(0, ...) לשמירת thread-safety

זרימת ניטור מקלדת

1. ההורה שולח: FORWARD|KEYLOG_START → השרת מעביר לילד
2. הילד מפעיל pynput Listener ב-thread נפרד
3. כל 20 הקשות או כל 0.8 שנייה – הלקוח שולח: <FORWARD|KEYLOG_DATA|<text
4. השרת שומר ב-clients_logs ומעביר להורה
5. בעצירה – השרת שומר לקובץ txt בתיקיית keylogs/ ומתעד בDB

2.4 פרוטוקול התקשורת

תיאור מבנה הפרוטוקול

הפרוטוקול מבוסס TCP, עם תוספת שכבת הצפנה AES-GCM מעל. כל הודעה נשלחת בשני שלבים:

- 4 bytes ראשונים: אורך ההודעה המוצפנת (big-endian integer)
- N bytes הבאים: ההודעה המוצפנת ב-Base64

בצד המקבל: קוראים 4 bytes, מחשבים N, קוראים N bytes, מפענחים AES-GCM, מקבלים את הטקסט המקורי.

רשימת הודעות המערכת

שם ההודעה	נשלחת מ-	נשלחת אל-	מבנה השדות
LOGIN	לקוח	שרת	`LOGIN|username|password`
LOGIN_OK	שרת	לקוח	`LOGIN_OK|role|sibling_status`
LOGIN_FAIL	שרת	לקוח	`LOGIN_FAIL` או `LOGIN_FAIL|סיבה`
REGISTER	לקוח	שרת	`[REGISTER|user|pass|role[|parent_id`
REGISTER_OK	שרת	לקוח	`REGISTER_OK|new_id`
REGISTER_INVALID_PARENT	שרת	לקוח	`REGISTER_INVALID_PARENT`
FORWARD|SCREEN_START	הורה	שרת → ילד	`FORWARD|SCREEN_START`
FORWARD|SCREEN_STOP	הורה	שרת → ילד	`FORWARD|SCREEN_STOP`
FORWARD|SCREEN_DATA	ילד	שרת → הורה	`<FORWARD|SCREEN_DATA|<base64_jpeg`
FORWARD|KEYLOG_START	הורה	שרת → ילד	`FORWARD|KEYLOG_START`
FORWARD|KEYLOG_STOP	הורה	שרת → ילד	`FORWARD|KEYLOG_STOP`
FORWARD|KEYLOG_DATA	ילד	שרת → הורה	`<FORWARD|KEYLOG_DATA|<text`
LOGOUT	לקוח	שרת	`LOGOUT`
REJECTED|SERVER_FULL	שרת	לקוח	`REJECTED|SERVER_FULL`
REJECTED|DDOS_DETECTED	שרת	לקוח	`REJECTED|DDOS_DETECTED`
LOGIN_FAIL|DDOS_BLOCKED	שרת	לקוח	`LOGIN_FAIL|DDOS_BLOCKED`

2.5 מסכי המערכת

מסך 1: מסך פתיחה (Splash Screen)

תפקיד: הצגת מסך פתיחה מרשים לפני הפעלת השרת.

תיאור: מסך מלא (fullscreen) עם וידאו ואודיו. אם אין וידאו, מוצגת אנימציה גרפית עם אייקון מגן, כיתוב PARENTAL CONTROL ובר התקדמות מונפש.

אינטראקציות: לחיצת ESC בלבד תסגור את המסך ותפעיל את השרת.

מסך 2: מסך כניסה (Login)

תפקיד: אימות זהות המשתמש.

תיאור: ממשק כהה ומקצועי עם שדות שם משתמש וסיסמה. כולל כפתור LOGIN, הודעות שגיאה אינליין (לא popup), וקישור למסך הרשמה.

אינטראקציות: הזנת פרטים + לחיצה על LOGIN או Enter. הפניה למסך הרשמה.

מסך 3: מסך הרשמה (Register)

תפקיד: יצירת חשבון חדש.

תיאור: דומה למסך הכניסה עם הוספת בחירת תפקיד (הורה/ילד). בחירת "ילד" חושפת שדה Parent ID.

אינטראקציות: הזנת פרטים + לחיצה על REGISTER. הפניה חזרה למסך הכניסה.

מסך 4: לוח בקרת הורה (Parent Dashboard)

תפקיד: מרכז השליטה של ההורה.

תיאור: ממשק דו-עמודי. עמודה שמאל – שידור מסך עם קנבס ישיר וכפתורי START/STOP. עמודה ימין – פאנל Keylogger עם כפתורי הפעלה ואזור תצוגת הקשות ירוקות על רקע כהה. שעון חי בפינה עליונה. מד סטטוס (IDLE/LIVE) על כל פאנל.

מסך 5: מסך ילד (Child Session)

תפקיד: הודעת 'סשן פעיל' לילד.

תיאור: כרטיס מרכזי עם נקודה ירוקה מהבהבת, כיתוב SESSION ACTIVE, הסבר קצר, ושעון חי.

תרשים מעברי מסכים

splash.py (שרת) → server.py מופעל → ממתין לחיבורים

client.py → [בדיקת קובץ creds] → אם נמצא: מצב headless → סיום

client.py → [לא נמצא קובץ] → מסך Login → הצלחה → Dash הורה / מסך ילד

מסך Login → [לחיצה על Register] → מסך Register → הצלחה → מסך Login

2.6 מבני נתונים ובסיס הנתונים

שם מסד הנתונים: project_maria

טבלה 1: clients

תיאור: מאחסנת את פרטי כל המשתמשים הרשומים במערכת.

שם שדה	טיפוס	מפתח?	תיאור
client_id	INT AUTO_INC	PRIMARY KEY	מזהה ייחודי אוטומטי
client_ip	VARCHAR(255)	—	כתובת IP של המשתמש
client_port	INT	—	פורט החיבור
client_username	VARCHAR(255)	—	שם המשתמש (ייחודי)
client_password	VARCHAR(255)	—	סיסמה מוצפנת SHA-256
client_role	VARCHAR(10)	—	'parent' או 'child'
client_sibling_id	INT	FK → client_id	מזהה הצמד
client_ddos_status	BOOLEAN	—	true אם חסום DDoS
client_total_audio_logs	INT	—	מונה לוגי שמע
client_total_keylogs	INT	—	מונה לוגי מקלדת

טבלה 2: keylogs

תיאור: מאחסנת מידע על קובצי יומן מקלדת שנשמרו בתיקיית /keylogs.

שם שדה	טיפוס	מפתח?	תיאור
keylog_id	INT AUTO_INC	PRIMARY KEY	מזהה ייחודי אוטומטי
keylog_parent_id	INT	FK → client_id	מזהה ההורה המנטר
keylog_child_id	INT	FK → client_id	מזהה הילד המנוטר
keylog_path_name	VARCHAR(255)	—	נתיב מלא לקובץ ה-txt

קבצי יומן מקלדת (/keylogs)

בנוסף לטבלת ה-DB, הנתונים עצמם נשמרים בקבצי טקסט בתיקיית /keylogs. שם כל קובץ: log_<child_id>_<YYYYMMDD_HHMMSS>.txt. הקובץ מכיל את כל ההקשות שהוקלטו, כולל תוספת timestamp בממשק ההורה.

2.7 סקירת חולשות ואיומי סייבר

שכבת האפליקציה

SQL Injection

איום: הזרקת קוד SQL דרך שדות שם משתמש/סיסמה כדי לעקוף אימות.

פתרון: כל שאילתות ה-SQL בפרויקט משתמשות בפרמטרים מוכנסים (%s placeholders) ולא בשרשור מחרוזות. בנוסף, הסיסמה עוברת hashing לפני כל שאילתה, כך שאפילו אם מוזרק קוד, ה-hash לא יתאים לאף ערך בDB.

(MITM (Man-In-The-Middle

איום: גורם שלישי ברשת שמיירט את התקשורת בין השרת ללקוח.

פתרון: כל הודעה מוצפנת AES-GCM לפני השליחה. GCM מספק גם confidentiality (אי-קריאות) וגם integrity (אי-שינוי). מפתח AES 256-bit קבוע משותף. מגבלה: מפתח קבוע (ולא דינמי per-session) – שיפור לגרסה עתידית.

DDoS / DoS

איום: הצפת השרת בחיבורים כדי לקרוס אותו או למנוע גישה למשתמשים לגיטימיים.

פתרון: המערכת מיישמת שני מנגנוני הגנה:

- מגבלת חיבורים גלובלית: MAX_TOTAL_CONNECTIONS=15. חיבור ה-16 נדחה עם REJECTED|SERVER_FULL
- מגבלה לפי IP: MAX_CONNECTIONS_PER_IP=3. חיבור ה-4 מאותו IP מפעיל ניתוק כל החיבורים מאותו IP + ddos_status=true בDB
- חסימה קבועה: משתמש עם ddos_status=true לא יכול להתחבר בכל ניסיון עתידי

אחסון סיסמאות

איום: גישה לא מורשית ל-DB תחשוף סיסמאות משתמשים.

פתרון: כל הסיסמאות מאוחסנות כ-SHA-256 hash בלבד. אף סיסמה לא נשמרת בטקסט גלוי. גם אם ה-DB נחשף, לא ניתן לשחזר את הסיסמאות המקוריות.

הפעלת המערכת

הגישה לשרת דורשת אימות (LOGIN) לפני כל פעולה. הסשן מנוהל ב-memory (מילון clients) ולא נשמר בקובץ. התנתקות מנקה את הרישום מיידית. פורט 9921 – ניתן לחסום בפיירוול לכניסות חיצוניות.

פרק 3: מימוש הפרויקט

3.1 סקירת מודולים ומחלקות

מודולים מיובאים עיקריים

שם המודול	מטרה
`socket`	תקשורת TCP/IP – יצירת חיבורים, שליחה וקבלת נתונים
`threading`	הרצת לקוחות וסשנים מרובים בו-זמנית
`tkinter`	ממשק משתמש גרפי – חלונות, כפתורים, קנבס
`(PIL (Pillow`	עיבוד תמונות – פתיחת JPEG, המרה ל-PhotoImage
`pyautogui`	צילום מסך תכנותי
`pynput`	האזנה להקשות מקלדת ברמת OS
`(cv2 (OpenCV`	קריאת קובץ וידאו frame-by-frame
`pygame`	ניגון קובץ MP3
`hashlib`	חישוב SHA-256 לסיסמאות
`base64`	קידוד תמונות לטרנסמיסיה טקסטואלית
`mysql.connector`	חיבור ופעולות בבסיס הנתונים
`Crypto.Cipher.AES`	הצפנה/פענוח AES-GCM

מחלקה: encryption.py (Encryption)

תפקיד: ניהול כל פעולות ההצפנה והפענוח של התקשורת.

פעולה	תיאור
`__init__()`	מאתחל מפתח AES 256-bit ו-Nonce קבוע 12 bytes
`encrypt_data(data: bytes) → str`	מצפין bytes, מחזיר Base64 + authentication tag
`decrypt_data(data: str) → bytes`	מפענח Base64+tag, מחזיר bytes מקוריים
`send_encrypted_message(sock, msg`	מצפין הודעה ושולח: 4 bytes אורך + הודעה מוצפנת
`receive_encrypted_message(sock) → str`	קורא 4 bytes אורך, קורא הודעה, מפענח ומחזיר string

מחלקה: db_manager.py (DatabaseManager)

תפקיד: ממשק אחיד לכל פעולות MySQL. מחסנת את פרטי החיבור ומנהלת קרסור לכל פעולה.

פעולה	תיאור
`connect_()`	יוצר חיבור MySQL עם הפרמטרים שנמסרו
`create_table(name, params)`	יוצר טבלה אם לא קיימת
`insert_row(table, cols, types, vals)`	מכניס שורה ומחזיר lastrowid
`get_rows_with_value(table, col, val)`	מחזיר שורות שמתאימות לתנאי
`update_row(table, pk_col, pk_val, cols, vals)`	מעדכן שורה לפי מפתח ראשי
`delete_row(table, col, val)`	מוחק שורות לפי ערך עמודה
`get_all_rows(table)`	מחזיר את כל השורות בטבלה
`close()`	סוגר את חיבור ה-DB

מחלקה: Server (server.py)

תפקיד: ניהול כל ההיגיון העסקי – קבלת חיבורים, ניתוב הודעות, ניהול DDoS, ממשק DB.

פעולה	תיאור
`start()`	מאזין על PORT, מקבל חיבורים, מפעיל thread לכל לקוח
`handle_client(sock, ip)`	לולאת טיפול בהודעות לקוח בודד
`register_connection(id, ip, sock)`	מוסיף חיבור לכל מבני הנתונים
`unregister_connection(id, ip)`	מסיר חיבור בסגירה/התנתקות
`check_global_limit()`	בודק אם הגענו ל-MAX_TOTAL_CONNECTIONS
`check_ip_limit(ip)`	בודק אם ה-IP הגיע ל-MAX_CONNECTIONS_PER_IP
`flag_ddos(ip)`	מנתק וחוסם את כל החיבורים מה-IP
`is_ddos_flagged(username)`	בודק ddos_status בDB

מחלקה: Client (client.py)

פעולה	תיאור
`connect()`	מתחבר לשרת, מחזיר True/False
`send(message)`	מצפין ושולח הודעה
`receive()`	מקבל ומפענח הודעה
`disconnect()`	שולח LOGOUT וסוגר socket

מחלקה: ClientGUI (client.py)

תפקיד: ניהול כל ממשק המשתמש הגרפי.

פעולה	תיאור
`build_login_screen()`	בונה את מסך הכניסה
`build_register_screen()`	בונה את מסך ההרשמה

build_parent_screen()	בונה את לוח הבקרה של ההורה
build_child_screen()	בונה את מסך הילד
listen_to_server()	thread מאזין – מעביר הודעות מהשרת
display_frame(photo_	מציג פריים מסך בקנבס (thread-safe)
update_log_display(text	מוסיף הקשות לתצוגת ה-keylogger
stream_screen_loop()	מצלם ושולח מסך בקצב 10fps
toggle_keylogger(start	מפעיל/עוצר pynput Listener
on_close_()	מנתק בצורה נקייה בסגירת חלון

3.2 אלגוריתמים מרכזיים

אלגוריתם 1: הצפנת AES-GCM

תיאור הבעיה: נדרש לשלוח נתונים רגישים (סיסמאות, נתוני מסך, הקשות מקלדת) דרך רשת, כך שמאזין ברשת לא יוכל לקרוא או לשנות אותם.

אלגוריתמים שנסקרו:

- RSA: הצפנה אסימטרית – מתאים להחלפת מפתחות, אבל איטי להצפנת נפחי נתונים גדולים כמו פריימי וידאו
- AES-CBC: מהיר, אבל אינו מספק authentication – תוקף יכול לשנות ciphertext
- AES-GCM: מהיר כמו CBC + מספק authentication tag – נבחר

תיאור הפתרון הנבחר: AES-GCM מצפין את הנתונים ומחשב tag אימות של 16 bytes. הכשלת בדיקת ה-tag מעידה על ניסיון שינוי. המפתח הוא 32 bits (256 bytes) ו-Nonce הוא 12 bytes כנדרש ב-GCM.

אלגוריתם 2: ניטול שידור המסך ללא ריצוד (Flicker-free)

תיאור הבעיה: שידור מסך דורש עדכון תמונה 10 פעמים בשנייה. גישה נאיבית של create_image() חוזרת בכל פריים יצרה שני בעיות: ריצוד (black frames) ודליפת זיכרון (כל PhotoImage נצבר).

אלגוריתמים שנסקרו:

- create_image בכל פריים – פשוט אבל יוצר ריצוד ודליפת זיכרון
- Canvas.itemconfig – מעדכן אובייקט קיים במקום ליצור חדש – נבחר

תיאור הפתרון הנבחר: נוצר אובייקט canvas image בודד בפריים הראשון. בכל פריים הבא מתבצעת קריאה ל-canvas.itemconfig(img_id, image=new_photo). כך Canvas מחזיק תמיד אובייקט אחד ואין ריצוד. ה-PhotoImage החדש נשמר ב-canvas._photo למניעת Garbage Collection.

אלגוריתם 3: Thread-Safety ב-Tkinter

תיאור הבעיה: הלקוח מקבל נתונים בthread רקע (listen_to_server), אך Tkinter מחייב שכל עדכוני UI יתבצעו מה-thread הראשי. גישה ישירה גרמה לקריסות שקטות.

פתרון: כל עדכון UI מה-background thread מתוזמן דרך root.after(0, fn, args). קריאה זו מוסיפה את הפונקציה לתור האירועים של Tkinter, שמבצע אותה ב-main thread. זוהי הגישה הסטנדרטית והבטוחה היחידה לניהול UI מרובה-threads ב-Python.

אלגוריתם 4: גילוי ומניעת DDoS

תיאור הבעיה: תוקף יכול לפתוח מאות חיבורים בו-זמנית כדי לשתק את השרת.

פתרון דו-שלבי:

1. בדיקה גלובלית: לפני קבלת כל חיבור חדש, נבדק total_connections. אם ≤ (MAX_TOTAL_CONNECTIONS 15) – הסוקט נדחה ונסגר מיד.
2. בדיקה לפי IP: אם ה-IP כבר מחזיק (MAX_CONNECTIONS_PER_IP 3) חיבורים – החיבור החדש נדחה, כל החיבורים הקיימים מאותו IP מנותקים, ו-ddos_status=true נשמר לDB לחסימה קבועה.

שני השלבים מוגנים ב-threading.Lock() למניעת race conditions.

3.3 מסמך בדיקות מלא

בדיקות שתוכננו בשלב האפיון

שם בדיקה	מטרה	מה בוצע	תוצאה	תיקון
כניסה תקינה	כניסה עם פרטים נכונים	login עם test_user_1/123	LOGIN_OK|child|NO_SI ✓ BLING	—
כניסה שגויה	סיסמה שגויה נדחית	login עם סיסמה לא נכונה	LOGIN_FAIL|Wrong ✓ password	—
הרשמת הורה	יצירת הורה חדש	REGISTER|user|pass|p arent	✓ <REGISTER_OK|<id	—
הרשמת ילד תקין	ילד מוצמד להורה	REGISTER|...|child|<par <ent_id	✓ מעודכן sibling_id	—
הרשמת ילד לא תקין	parent_id שגוי נדחה	REGISTER|...|child|999	REGISTER_INVALID_P ✓ ARENT	—
שידור מסך	פריימים מגיעים להורה	FORWARD|SCREEN_S TART	✓ תמונות בקנבס	תוקן: itemconfig במקום create_image
עצירת שידור	שידור עוצר	FORWARD|SCREEN_S TOP	✓ קנבס מתנקה	—
ניטור מקלדת	הקשות מגיעות	FORWARD|KEYLOG_S TART + הקלדה	✓ טקסט בממשק	תוקן: root.after() לthread-safety
שמירת keylog	קובץ נוצר	FORWARD|KEYLOG_S TOP	✓ log_X_timestamp.txt	תוקן: נתיב /keylogs
DDoS גלובלי	לקוח 16 נדחה	16 חיבורים בו-זמנית	REJECTED|SERVER_F ✓ ULL	—
DDoS לפי IP	IP עם 4 חיבורים חסום	4 לקוחות מאותו IP	✓ ddos_status=true	—
חסימת DDoS	משתמש חסום לא נכנס	login לאחר חסימה	LOGIN_FAIL|DDOS_BL ✓ OCKED	—
threads 20	multi_run עובד	הרצת multi_run.py	✓ רצים threads 20	—
REGISTER+role	register בלי role נכשל	REGISTER|u|p (ללא role)	role :תוקן — IndexError חובה	הוסף role=parent כ-default

בעיות עיקריות שנתגלו ותוקנו

1. בעיית Thread-Safety בממשק: שידור המסך והמקלדת לא עבדו כי הthread הרקע ניסה לעדכן Tkinter ישירות. תוקן על ידי העברת כל עדכוני UI דרך root.after(0, fn).

2. ריצוד בשידור המסך: create_image() חוזרת יצרה פריימים שחורים ביניהם. תוקן על ידי שימוש ב-itemconfig על אובייקט קיים.

3. REGISTER ללא role: הלקוח שלח 3 שדות, השרת ציפה ל-4. תוקן בלקוח ובקבצי ה-creds.

4. נתיב שגוי לאודיו: pygame לא מצא את splash_audio.mp3 כי ה-path חושב ממיקום שגוי. תוקן על ידי os.path.abspath(__file__).

5. סיסמת DB חסרה: 2wsx3edc4rfv במקום 2wsx3edc4rfv. – גרמה לכשל חיבור. תוקן.

פרק 4: מדריך למשתמש

4.1 עץ קבצי המערכת

/PROJECT_MARIA

server.py ── ├ ← הפעלת השרת (כולל splash)

client.py ── ├ ← הפעלת לקוח

splash.py ── ├ ← מסך פתיחה (מופעל אוטומטית)

constants.py ── ├ ← הגדרות כלליות

encrypt.py ── ├ ← מחלקת הצפנה

db_manager.py ── ├ ← מחלקת DB

db_tools.py ── ├ ← כלי DB נוספים

create_tables.py ── ├ ← יצירת טבלאות

tools_no_encryption.py ── ├ ← כלי עזר (hash)

multi_run.py ── ├ ← הרצת 20 לקוחות

audio_test.py ── ├ ← אבחון אודיו

splash.mp4 ── ├ ← וידאו מסך פתיחה

splash_audio.mp3 ── ├ ← אודיו מסך פתיחה

creds_1.txt … creds_10.txt ── ├ ← פרטי כניסה סטטיים

/keylogs ── └ ← תיקיית יומני מקלדת

log_<id>_<timestamp>.txt ── └

4.2 דרישות מערכת והתקנה

דרישות מינימום

- מערכת הפעלה: Windows 10/11 (64-bit)
- Python: 3.8 ומעלה
- RAM: 4GB לפחות
- רשת: LAN / Wi-Fi משותף לשרת ולכל הלקוחות
- MySQL/MariaDB: גרסה XAMPP.8 (או מומלץ)

התקנת חבילות Python

pip install opencv-python pillow pygame pyautogui pynput pycryptodome mysql-connector-python

הגדרת בסיס הנתונים

3. הפעל את XAMPP ולחץ Start על MySQL
4. פתח (phpMyAdmin (http://localhost/phpmyadmin
5. צור מסד נתונים בשם project_maria
6. ודא שמשתמש root עם הסיסמה הנכונה קיים
7. הפעל את create_tables.py פעם אחת בלבד

4.3 הפעלת השרת

הרץ מה-PowerShell / Command Prompt:

python server.py

מסך הפתיחה יופיע אוטומטית. לאחר סגירתו, השרת יתחיל להאזין. תראה בטרמינל:

Server listening on 127.0.0.1:9921

Limits — Max total: 15, Max per IP: 3

4.4 הפעלת לקוח – מצב אינטראקטיבי (GUI)

ודא שאין קובץ creds_*.txt תקין בתיקייה, ואז הרץ:

python client.py

ממשק הכניסה יפתח. הזן שם משתמש וסיסמה ולחץ LOGIN (או Enter). לאחר כניסה מוצלחת תוצג ממשק הורה או ילד בהתאם לתפקיד.

4.5 הפעלת לקוח – מצב אוטומטי (Headless)

אם קיים קובץ creds_X.txt תקין בתיקייה, הלקוח ירוץ אוטומטית ללא ממשק. הרץ:

python client.py

פלט הטרמינל יראה: [Headless] LOGIN succeeded for 'test_user_3'. Response: LOGIN_OK|child|NO_SIBLING

4.6 הפעלת 20 לקוחות בו-זמנית

python multi_run.py

20 threads יופעלו בו-זמנית. כל thread מפעיל עותק של client.py ללא ממשק. הפלט יראה מספר Thread וסטטוס לכל אחד. הבדיקה הזו מאמתת שהשרת עמיד תחת עומס.

4.7 הפעלת ניטור מסך ומקלדת

8. הורה וילד מחוברים למערכת (LOGIN_OK לשניהם)
9. בממשק ההורה, לחץ ▶ START STREAM – שידור המסך יתחיל
10. לחץ ⏺ START LOG – ניטור המקלדת יתחיל
11. ההקשות יופיעו בזמן אמת בפאנל הימני, עם timestamp

12. לחץ 🔲 STOP LOG – הניטור נעצר והיומן נשמר ב-keylogs/
13. לחץ ■ STOP STREAM – השידור נעצר

פרק 5: רפלקציה וסיכום אישי

5.1 תהליך העבודה על הפרויקט

עבודה על הפרויקט הזה הייתה מסע ארוך ומאתגר שנמשך כמעט שנה שלמה. כאשר התחלתי, ידעתי לכתוב פרוגרמות Python בסיסיות, אבל לא הייתה לי כמעט שום הבנה של תקשורת רשת, ממשקי משתמש גרפיים מורכבים, בסיסי נתונים, או הצפנה. כל אחד מהתחומים הללו דרש ממני ללמוד מאפס.

השלב הראשון והמשמעותי ביותר היה להבין את מודל ה-Client-Server. בתחילה ניסיתי לבנות קשר ישיר בין שני לקוחות – ללא שרת אמצעי – וגיליתי שזה מסרבל מאוד את ניהול ה-NAT ואינו ניתן לסקייל. המעבר לארכיטקטורת שרת מרכזי היה ההחלטה הטובה ביותר שקיבלתי בפרויקט.

שלב ההצפנה היה מאתגר במיוחד. בתחילה לא הבנתי את ההבדל בין CBC ל-GCM. לאחר קריאת תיעוד ומאמרים, הבנתי ש-GCM מוסיף authentication tag שמגן מפני שינוי ה-ciphertext – תכונה חיונית למניעת MITM.

5.2 ההצלחות

- שידור מסך חי: ראיתי בפעם הראשונה את מסך מחשב אחד מוצג בזמן אמת על מחשב שני – זה היה רגע מרגש מאוד
- הצפנה עובדת: Wireshark הראה שהנתונים ברשת לא קריאים – ההצפנה עובדת
- threads 20 בו-זמנית: multi_run.py עם 20 threads רץ ללא קריסה
- GUI מקצועי: הממשק הכהה והמעוצב קיבל ציון גבוה מהמורה ומחברי
- מנגנון DDoS: המנגנון זיהה וחסם IP חריג בהצלחה

5.3 האתגרים והקשיים

הקושי הגדול ביותר היה בעיית ה-Thread-Safety. שבוע שלם לא הצלחתי להבין למה שידור המסך עובד אחת לכמה פעמים ולפעמים כלל לא. לאחר שימוש ב-debugger גיליתי שהthread הרקע מנסה לעדכן Tkinter ישירות – דבר שאסור. הפתרון דרך root.after(0, ...) היה פשוט, אבל מצאתי אותו רק לאחר שהבנתי את הבעיה לעומק.

בעיה נוספת הייתה הריצוד בשידור המסך. create_image() חוזרת הצטברה ויצרה מאות אובייקטים ב-Canvas. המעבר ל-itemconfig() פתר את הבעיה תוך דקות – אבל הגעתי לפתרון רק לאחר שהתייעצתי עם Claude AI שהסביר לי את הגישה.

גם ניהול הסיסמה השגויה בקוד (2wsx3edc4rfv במקום 2wsx3edc4rfv.) הסב לי שעות של debugging – טעות קטנה עם השפעה גדולה.

5.4 תהליך הלמידה

הפרויקט לימד אותי יותר מכל קורס אחד. הנושאים החדשים שלמדתי:

- תקשורת TCP/IP וניהול Sockets ב-Python
- הצפנת AES-GCM וביצוע authenticated encryption
- Threading מתקדם וThread-Safety
- SQL ועבודה עם MySQL דרך Python

- עיצוב GUI מקצועי עם Tkinter
- עיבוד תמונה וקידוד Base64 לשידור
- שימוש ב-OpenCV ו-pygame
- ניהול פרויקט תוכנה: תכנון, מימוש, בדיקות, תיעוד

5.5 כלים שנלקחים להמשך

מהפרויקט הזה אני לוקח איתי כלים מקצועיים שיישמשו אותי לכל החיים: היכולת לבנות מערכת Client-Server מלאה, ידע בהצפנה יישומית, הבנה של Thread-Safety, ועבודה עם בסיסי נתונים. אבל מעל לכל – למדתי לפרק בעיה מורכבת לחלקים קטנים, לפתור כל חלק בנפרד, ולחבר הכל לפתרון שלם.

5.6 בראייה לאחור

אם הייתי מתחיל מחדש, הייתי:

- מיישם מפתחות הצפנה דינמיים (Diffie-Hellman) במקום מפתח קבוע
- מוסיף logging מקיף יותר לשרת לאבחון קל יותר
- כותב unit tests מהיום הראשון ולא רק בסוף
- משתמש ב-asyncio (async/await) במקום threads – יעיל יותר

5.7 שיפורים עתידיים אפשריים

- הצפנה דינמית: מפתח Diffie-Hellman ייחודי לכל סשן
- ניטור שמע: הוספת מיקרופון לניטור
- ממשק Web: פאנל ניהול דרך דפדפן
- תמיכה ב-Linux ו-macOS
- התראות: SMS/Email להורה בעת גילוי פעילות חשודה
- הקלטת מסך: שמירת וידאו ולא רק זרם חי

5.8 תודות

ברצוני להודות למורה המנחה שלי על הכוונה לאורך כל השנה, ועל הסבלנות בהסבר נושאים מורכבים. תודה גם לחבריי לכיתה שאיתם שיתפתי קשיים ופתרונות – למידת עמיתים הייתה משמעותית מאוד בפרויקט הזה. תודה מיוחדת ל-Claude AI שסייע לי בהבנת בעיות thread-safety ועיצוב ה-GUI.

פרק 6: ביבליוגרפיה

Python Software Foundation. (2024). socket — Low-level networking interface. Python Documentation. https://docs.python.org/3/library/socket.html

Python Software Foundation. (2024). threading — Thread-based parallelism. Python Documentation. https://docs.python.org/3/library/threading.html

Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. Python Documentation. https://docs.python.org/3/library/tkinter.html

Legrandin. (2024). PyCryptodome Documentation — AES-GCM Mode. https://pycryptodome.readthedocs.io/en/latest/src/cipher/modern.html#gcm-mode

Pillow Developers. (2024). Pillow (PIL Fork) Documentation. https://pillow.readthedocs.io

Moses, A. (2024). PyAutoGUI Documentation. https://pyautogui.readthedocs.io

Hammarberg, J. (2024). pynput Documentation. https://pynput.readthedocs.io

OpenCV Team. (2024). OpenCV – Open Source Computer Vision Library. https://opencv.org

Pygame Community. (2024). Pygame Documentation. https://www.pygame.org/docs

Oracle Corporation. (2024). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en

NIST. (2007). Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM). NIST Special Publication 800-38D. https://csrc.nist.gov/publications/detail/sp/800-38d/final

OWASP Foundation. (2024). SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

פרק 7: נספחים

נספח א': קובץ constants.py

קובץ זה מכיל את כל הקבועים הגלובליים של המערכת:

קבוע	ערך	משמעות
`CHUNK_SIZE`	4096	גודל chunk בקריאת socket
`IP`	127.0.0.1	כתובת IP של השרת
`PORT`	9921	פורט ההאזנה
`MAX_TOTAL_CONNECTIONS`	15	מגבלת חיבורים כוללת
`MAX_CONNECTIONS_PER_IP`	3	מגבלת חיבורים לכל IP
`DB_CONFIG`	host/user/pass/db	פרמטרי חיבור MySQL

נספח ב': מבנה קבצי פרטי כניסה (creds_*.txt)

פורמט: [ACTION|username|password[|role

דוגמאות:

creds_1.txt → LOGIN|test_user_1|123

creds_9.txt → REGISTER|bot_user_9|pass9|parent

creds_7.txt → LOGIN|test_user_1|wrong_password

נספח ג': הודעות שגיאה נפוצות

הודעה	סיבה ופתרון
`Cannot connect to server`	השרת לא פועל. הרץ server.py תחילה
`IndexError: list index out of range`	קובץ creds חסר שדה. ודא פורמט ACTION|user|pass|role
`ACCESS DENIED for root`	סיסמת MySQL שגויה ב-constants.py
`REGISTER_INVALID_PARENT`	parent_id לא קיים או כבר יש לו ילד
`LOGIN_FAIL|DDOS_BLOCKED`	ה-IP או המשתמש חסומים. אפס ddos_status בDB
`splash_audio.mp3 not found`	הקובץ חסר בתיקייה. ודא שהוא נמצא ב-PROJECT_MARIA/
`pygame not installed`	הרץ: pip install pygame

נספח ד': ניהול סיכונים

סיכון	חומרה	תכנון ההתמודדות	מה בוצע בפועל
כשל Thread-Safety	גבוהה	להשתמש ב-locks	root.after(0,...) לכל UI update
גישה לDB נכשלת	בינונית	try/except + reconnect	create_all_tables בהפעלה ראשונה
מתקפת DDoS	גבוהה	מגבלות IP וגלובלי	flag_ddos + ddos_status_ בDB
חשיפת סיסמאות	גבוהה	הצפנת Hash	SHA-256 על כל סיסמה
ריצוד בשידור	בינונית	כתיבה יעילה של Canvas	itemconfig במקום create_image
חיבור רשת נפסק	בינונית	exception handling	connected=False + finally cleanup